

Advanced Lesson 12 - Exercises: Pure Functions

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: Applying pure functions to values

```
Clear[x, f, data, newData];
```

```
(#12 &) [3]
```

```
(Sin[#1] &) [ $\frac{\pi}{2}$ ]
```

```
({#1, Sin[#1]} &) [ $\frac{\pi}{2}$ ]
```

```
({#1, Sin[#2]} &) [ $1, \frac{\pi}{2}$ ]
```

```
(#1 ≥ 10 &) [12]
```

```
(#1 ≥ 10 &) [8]
```

```
(#1 ≥ #2 &) [12, 8]
```

```
(#1 ≥ #2 &) [8, 12]
```

A pure function is a function with no name. The ampersand marks where the body ends, and the slots #1, #2 and so on stand for the arguments; a bare # is shorthand for #1.

The results are 9; 1; {Pi/2, 1}; {1, 1}; True; False; True; False.

The fourth is worth a second look: two arguments were supplied, the first slot took 1 and the second took Pi/2, and the body used them in the order written rather than the order given. Slots are addressed by number, not consumed in sequence.

The ampersand binds very loosely, which is why the bodies above are

parenthesised. Without the parentheses, $\#^2 \& [3]$ would parse as the pure function applied to nothing useful.

2: Selecting the points at most 20

```
data = {4, 65, 12, 4, 8, 21, 4};
Select[data, #1 ≤ 20 &]
```

Select takes a list and a predicate - a function returning True or False - and keeps the elements for which the predicate is True, in their original order. The result is {4, 12, 4, 8, 4}. Duplicates are kept; Select filters, it does not deduplicate.

3: Selecting the points between 5 and 20 inclusive

```
Select[data, 5 ≤ #1 ≤ 20 &]
```

The result is {12, 8}. Chained inequalities work directly in a pure function body, and are clearer than writing an explicit conjunction of two tests.

4: Mapping to ordered pairs

```
Clear[f];
({#1, f[#1]} &) /@ data
```

Map applies the function to each element and returns the list of results, so the output is a list of seven pairs: {4, f[4]}, {65, f[65]}, and so on. f is undefined, so the second component of each pair stays symbolic - which is the honest result, and exactly the shape ListPlot would want once f is given a definition.

5: Selecting points whose first coordinate squared is at most the second

```
newData = {{1, 4}, {3, 2}, {2, 2}, {5, 1}, {2, 5}};
Select[newData, #1[[1]]^2 ≤ #1[[2]] &]
```

Here each element of the list is itself a pair, so the slot stands for a whole pair and Part is used inside the pure function to reach its components. Checking the five pairs in turn: 1 squared is 1 and $1 \leq 4$, so {1, 4} is kept; $9 > 2$, so {3, 2} is dropped; $4 > 2$, so {2, 2} is dropped;

$25 > 1$, so $\{5, 1\}$ is dropped; $4 \leq 5$, so $\{2, 5\}$ is kept. The result is $\{\{1, 4\}, \{2, 5\}\}$.

6: Derivatives as pure functions

```
Clear[f, x];
```

```
f'[x]
```

```
f'
```

```
Sin'[x]
```

```
Sin'
```

Evaluating one line at a time. $f'[x]$ returns $f'[x]$ itself, printed as `Derivative[1][f][x]` in full form - Mathematica knows nothing about f , so it keeps the derivative symbolic.

f' returns the operator alone, `Derivative[1][f]`: the derivative of f as a function, not evaluated at any point.

$\text{Sin}'[x]$ returns $\text{Cos}[x]$, because the derivative of the built-in sine is known.

Sin' returns `Cos[#1] &`. That is the point of putting this exercise in the pure-function lesson. Differentiation in Mathematica is an operator that maps a function to a function, and when the result has no built-in name it comes back as an anonymous function - a pure function, produced by the system rather than typed by the user. `Cos[#1] &` and `Cos` are the same function; only one of them has a name.